

AMC feedback loop

Technical design v2 — end-to-end flow

AMC codebase capture → Human Feedback Loop module (unified record) → external Evaluation Layer (batch + tiered, repair)

v2 draft · three-system architecture: AMC, Human Feedback Loop module, Evaluation Layer

1. End-to-end flow

Three separate systems are involved. **AMC** owns the query/response flow, the feedback UI panel, and the `feedback_records` table. The **Human Feedback Loop module** is a separate codebase that builds one unified evaluation record per response. The **Evaluation Layer** is a separate system again, triggered per record, that does the actual deep evaluation and repair — its design is covered in Section 4 and is expected to change.

1.1 Capturing feedback in AMC

The response panel offers issue-type checkboxes and a feedback text box next to every AMC response. Active and passive capture both write to the same table; the difference is what triggers the write and how much of the record is populated.

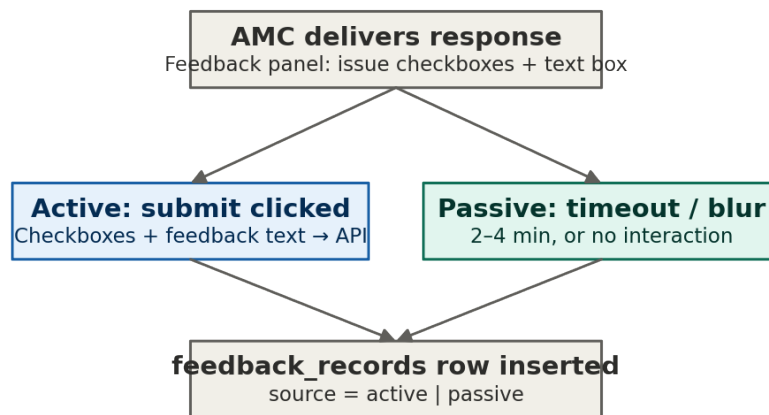


Figure 1 — active and passive capture into `feedback_records`

- **Active:** user checks one or more issue types, optionally adds free text, clicks Submit — sent via the feedback API immediately.
- **Passive:** a 2–4 minute window is given for the user to react. If the window elapses with no submit, or a blur/click-away event fires on the feedback box first, a row is captured with whatever partial state exists (often nothing — the response was accepted silently).

1.2 Handoff to the Human Feedback Loop module

Once a `feedback_records` row exists (from either path), AMC triggers the module — this is an active call, not the module polling AMC's database, since the two are separate codebases and AMC owns the source table.

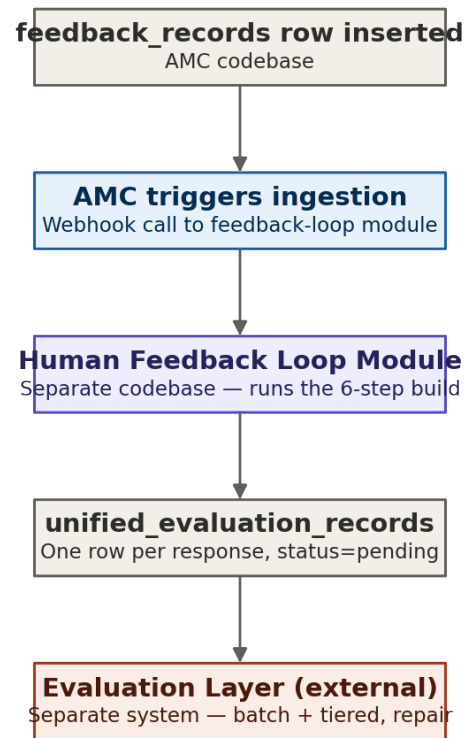


Figure 2 — trigger through to the external evaluation layer

The module runs the same six-step build for both active and passive rows, skipping steps that need data the passive path doesn't have (Section 3). The result is one JSON record in `unified_evaluation_records`, which the external Evaluation Layer then picks up independently.

2. AMC codebase changes

Everything in this section lives in the existing AMC / streamlit_app codebase. Nothing here depends on the Human Feedback Loop module's internals — only on its trigger contract.

2.1 UI panel

- Rendered alongside every delivered response.
- Issue-type checkboxes (multi-select) — e.g. Inaccurate, Incomplete, Irrelevant, Outdated, Other.
- A feedback text box for free-form detail.
- One Submit button. Clicking it is the active-path trigger; the panel also arms the passive-path timer/blur listener the moment the response renders, regardless of whether the user ever touches it.

2.2 feedback_records schema

Column	Type	Notes
session_id	text	Existing session identifier
response_id	text	Existing response identifier, FK to the delivered response
issue_types	text[]	Null on passive-timeout rows with no interaction
feedback_text	text	Null on passive-timeout rows with no interaction
source	text	'active' 'passive'
created_at	timestampz	When this row was written
ingestion_status	text	'pending' 'sent' 'failed' — tracks the trigger call in 2.3

2.3 Trigger to the Human Feedback Loop module

Immediately after the insert (either path), AMC calls the module's ingestion endpoint with just the identifiers — the module fetches the rest itself via its own EvidenceAdapter, keeping AMC's payload minimal and avoiding a second source of truth for the feedback content.

```
POST https://feedback-loop.internal/ingest
{
  "session_id": "...",
  "response_id": "...",
  "source": "active" | "passive"
}
```

Decision to confirm: a webhook call can fail or the module can be briefly down. Recommend AMC sets ingestion_status='pending' on insert, flips to 'sent' on a 2xx response, and a lightweight retry sweep (e.g. every 5 minutes) re-sends anything still 'pending' or 'failed' after N minutes — so a dropped webhook call doesn't silently lose a feedback row.

3. Building the unified evaluation record

This is the Human Feedback Loop module's only job: turn one `feedback_records` row into one `unified_evaluation_records` row. Six steps run in sequence, each adding to the same JSON object; nothing is persisted until the last applicable step finishes.

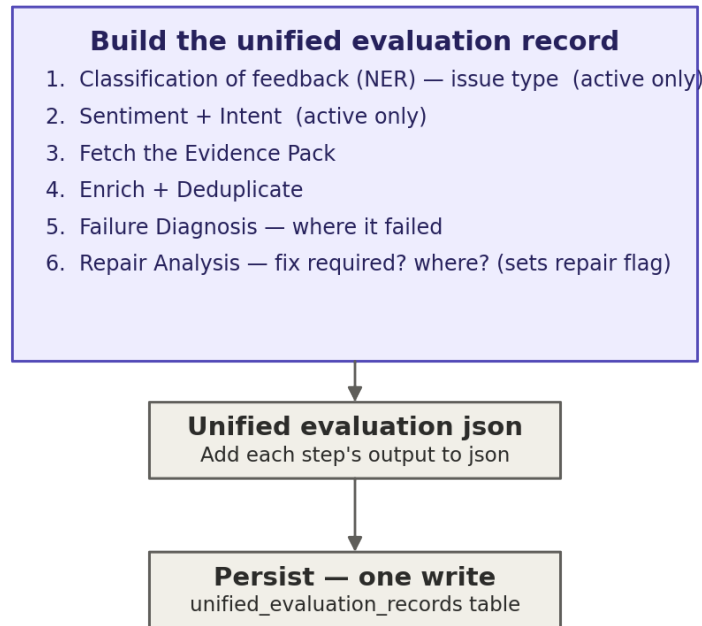


Figure 3 — the 6-step build, one JSON object, one write

What each step does, and what it needs

- **1 — Classification (NER):** extracts entities/issue type from the feedback text and checkbox selections. *Active only* — nothing to classify on a silent passive row.
- **2 — Sentiment + Intent:** scores tone and intent from the same feedback text. *Active only*, same reason as step 1.
- **3 — Fetch the Evidence Pack:** pulls the frozen D0–D3 evidence for this `response_id` from AMC's own store, via the read-only `EvidenceAdapter`. Runs for *both* paths — a passive row still needs the evidence to diagnose anything.
- **4 — Enrich + Deduplicate:** clusters this feedback against other open feedback on the same response/entity; a duplicate match short-circuits the remaining steps.
- **5 — Failure diagnosis:** determines whether and where the response failed, using whatever's available — evidence alone on a silent passive row, evidence plus classification on an active one.
- **6 — Repair analysis:** decides if a fix is required and, if so, where (vector store vs. graph vs. neither) — sets a boolean `repair` flag and a target. This step does not apply anything; it only flags for the Evaluation Layer.

Active vs. passive: what actually runs

Step	Active	Passive
1. Classification (NER)	Runs	Skipped — no text
2. Sentiment + Intent	Runs	Skipped — no text
3. Fetch Evidence Pack	Runs	Runs
4. Enrich + Deduplicate	Runs	Runs
5. Failure diagnosis	Runs	Runs (evidence-only)
6. Repair analysis	Runs	Runs

The function that builds the record

```
class UnifiedEvaluationRecord(BaseModel):
    session_id: str
    response_id: str
    created_at: datetime
    source: Literal["active", "passive"]
    status: Literal["pending", "in_progress", "resolved"] = "pending"

    classification: FeedbackClassification | None = None    # step 1, active only
    sentiment_intent: SentimentIntent | None = None        # step 2, active only
    evidence_pack: EvidencePack | None = None              # step 3
    enrichment: EnrichmentContext | None = None            # step 4
    diagnosis: FailureDiagnosis | None = None              # step 5
    repair: bool = False                                   # step 6
    repair_target: Literal["vector", "graph", None] = None # step 6

def build_unified_record(row, adapters, db) -> UnifiedEvaluationRecord | None:
    record = UnifiedEvaluationRecord(session_id=row.session_id, response_id=row.response_id,
                                     created_at=now(), source=row.source)

    # Steps 1-2 – active path only
    if row.source == "active" and row.feedback_text:
        record.classification = classify_feedback_ner(row.feedback_text, row.issue_types)
        record.sentiment_intent = score_sentiment_intent(row.feedback_text)

    # Step 3 – both paths
    record.evidence_pack = adapters.evidence.get_evidence_pack(row.response_id)

    # Step 4 – both paths
    record.enrichment = enrich_and_deduplicate(row, record.evidence_pack, db)
    if record.enrichment.is_duplicate:
        return None # already covered by an existing cluster

    # Step 5 – both paths, degrades gracefully without classification
    record.diagnosis = diagnose_failure(record.evidence_pack, record.classification)

    # Step 6 – both paths
    repair_needed, target = analyze_repair(record.diagnosis)
    record.repair = repair_needed
    record.repair_target = target

    return record

def on_ingest(session_id, response_id, source, adapters, db):
    """Called by AMC's webhook, or by the retry sweep for a failed delivery."""
    row = db.fetch_feedback_record(session_id, response_id)
    record = build_unified_record(row, adapters, db)
    if record:
        db.write_unified_record(record) # single write, one row per response
```

4. Evaluation layer (evaluation and repair)

This section reflects IMPLEMENTATION_STEPS_BATCH_TIERED.html as supplied and is expected to change. It's included here to show the intended contract with Section 3's output, not as a finalized spec.

This system is managed outside the Human Feedback Loop module's codebase. It is triggered independently for each `unified_evaluation_records` row where `repair = true`, and owns two engines: a passive-path batch LLM evaluator, and an active/HITL tiered ladder that escalates only when a cheaper tier can't resolve the claim.

4.1 Batch Evaluation Engine (passive path)

Scores feedback in batches of 25 under seven token-optimisation layers, and proposes graph corrections as a validated Cypher mutation rather than applying anything directly.

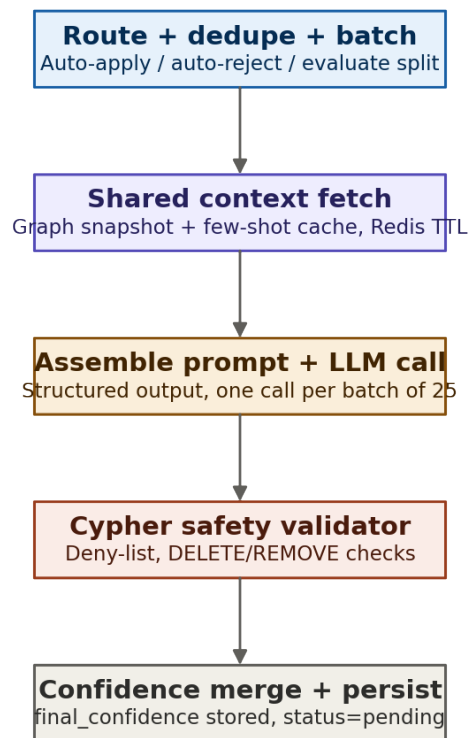


Figure 4 — batch evaluation engine pipeline

- **Route:** quality score ≥ 0.85 and inter-annotator agreement ≥ 0.90 auto-applies without an LLM call; below 0.20 auto-rejects; everything else queues for batching.
- **Dedupe + batch:** embeds and clusters claims (cosine ≥ 0.88), batches the deduplicated queue in groups of 25.
- **Shared context:** one Neo4j round-trip per batch (1-hop subgraph, Redis-cached 1h TTL) and a few-shot example cache (24h TTL) — both amortised across the whole batch instead of per item.

- **Cypher safety validator:** deny-lists DROP/CREATE INDEX/CREATE CONSTRAINT, rejects unguarded DELETE, rejects REMOVE on compliance-sensitive fields. A rejection is marked `insufficient_evidence` and logged, never silently dropped.
- **Confidence merge:** $\text{final_confidence} = \text{llm_confidence} \times \text{inter_annotator_agreement}$, stored on every item regardless of score — the apply-time gate is a separate decision made later.

4.2 Tiered Evaluation Engine (active / HITL path)

A four-tier ladder, cheapest first. Execution stops at the first tier that resolves the claim — a more expensive tier never runs once a cheaper one is confident.

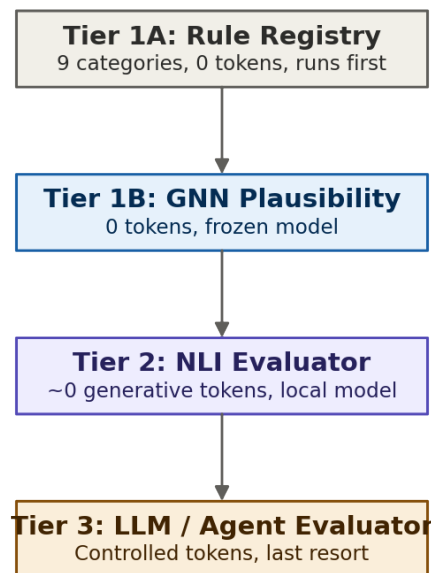


Figure 5 — the four-tier evaluation ladder

Tier	What it does	Cost
1A — Rule Registry	9 deterministic categories (schema, cardinality, referential, temporal, numeric, cross-source, duplicate, hierarchy, jurisdiction)	0 tokens
1B — GNN Plausibility	Structural plausibility score from a frozen model; ships as a 0.5 stub until trained	0 tokens
2 — NLI Evaluator	Local cross-encoder; entailment > 0.7 resolves, contradiction/neutral > 0.7 escalates	~0 generative tokens
3 — LLM / Agent Evaluator	Minimal evidence packet (claim + top-2 chunks + prior tier findings), last resort	Controlled tokens

Escalates to human review when all three scored tiers return confidence below 0.60, the attribute is compliance-sensitive, or two authoritative sources conflict.

Tier 1B build options

Both share the same `GNNPlausibilityScorer.score()` interface, so the choice is an internal swap. Recommended starting point: Option 2 (GDS), since it needs zero new runtime dependencies and runs inside the Neo4j instance already in place; swap to Option 1 later if finer control over the architecture is needed.

	Option 1 — Python R-GCN	Option 2 — Neo4j GDS
Cost	Free (open-source)	Free (GDS Community Edition)
New infra	New PyTorch dependency + offline training job	None — runs inside existing Neo4j
Control	Full — custom architecture, features, loss	Bounded to the GDS pipeline API
Team lift	Requires PyTorch/PyG familiarity	Requires Cypher + GDS familiarity

Appendix: edge cases and open decisions

- **Repair locking (TBD):** if the same underlying issue is flagged by multiple concurrent records while a fix is already in progress, later repair attempts need to be locked or merged against the in-flight one rather than racing it. Locking key, scope (entity-level? recipe-level?), and timeout are not yet designed.
- **Trigger delivery guarantee:** the webhook-plus-retry-sweep approach in Section 2.3 is a recommendation, not yet confirmed — an alternative is a message queue between AMC and the module for at-least-once delivery without a custom retry sweep.
- **Evaluation Layer contract stability:** Section 4 is explicitly subject to change; the one contract Section 3 depends on is the shape of a `unified_evaluation_records` row with `repair=true` and a `repair_target` — worth keeping stable even as the engine internals evolve.